Universidad Internacional del Ecuador

Carrera: Ingeniería en Software Modalidad En Línea
Materia: PROGRAMACION ORIENTADA A OBJETOS
Estudiante: SALAZAR CADENA JHONNATAN FRANCISCO
Fecha: 15 de febrero del 2026

Aprendizaje Autónomo 2: Desarrollo del Sistema de Gestión de Libros Electrónicos

Docente: PALACIOS MOROCHO MILTON RICARDO

Tabla de contenido

Aprendizaje Autónomo 2: Desarrollo del Sistema de Gestión de Libros Electrónicos. 1
Introducción ... 3
Objetivo del Aprendizaje Autónomo 2 ... 3
Descripción general del sistema ... 3
PASO 1: Continuación del plan ... 4
Inicio de la codificación del sistema ... 4
Aplicación de los temas de las Unidades 1 y 2 ... 5
Identificación de clases (estructuras) y documentación de su uso ... 6
Entidad Libro ... 6
Lógica de negocio (Catalog) ... 7
Persistencia de datos ... 8
Avance significativo del desarrollo del software ... 9
PASO 2: Desarrollo del sistema ... 10
Implementación de los temas de la Unidad 3 (Interfaces) ... 10
Encapsulación ... 11
Manejo de errores ... 12
Comentarios y documentación del código ... 13
Conclusiones ... 13
URL web repositorio de Github ... 14

Introducción

El presente documento describe de manera detallada el progreso realizado en el desarrollo del Sistema de Gestión de Libros Electrónicos como parte del Aprendizaje Autónomo 2 de la asignatura Programación Orientada a Objetos. Este trabajo se enfoca en la aplicación práctica de los conocimientos teóricos revisados durante el curso, especialmente en el uso de estructuras de datos, principios de programación orientada a objetos, encapsulación, manejo de errores e implementación de interfaces.

A diferencia del Aprendizaje Autónomo 1, donde el objetivo principal fue construir un sistema funcional, en esta segunda etapa el énfasis se centró en mejorar la arquitectura del software, su organización interna y su capacidad de mantenimiento y escalabilidad. Para ello, se realizaron cambios estructurales en el código, se reorganizó el proyecto en capas bien definidas y se implementaron mecanismos de desacoplamiento mediante interfaces, cumpliendo con los requerimientos establecidos para este aprendizaje autónomo.

Objetivo del Aprendizaje Autónomo 2

El objetivo del Aprendizaje Autónomo 2 es aplicar estructuras de datos en la resolución de problemas a través de una revisión teórico-práctica, permitiendo implementar adecuadamente interfaces dentro de un entorno de programación orientado a objetos. Asimismo, se busca reforzar la encapsulación, el manejo de errores y la documentación del código, evidenciando un avance significativo del sistema desarrollado.

Descripción general del sistema

El sistema desarrollado permite gestionar libros electrónicos mediante funcionalidades como el registro de nuevos libros, el listado completo, la búsqueda por distintos criterios, la edición de información existente, la eliminación de registros y el cambio de disponibilidad. Además, incorpora persistencia de datos en formato JSON, lo que permite conservar la información entre ejecuciones.

La interacción con el usuario se realiza a través de un menú por consola, diseñado para validar las entradas del usuario y mostrar mensajes claros ante cualquier error o acción realizada. Este sistema fue seleccionado debido a su capacidad de crecer en complejidad y por permitir aplicar de forma integral los conceptos revisados en la asignatura.

```
Problems   Output   Debug Console   Terminal   Ports

PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos> go run ./cmd/app
=====================================
  SISTEMA DE GESTIÓN DE E-BOOKS (GO)
=====================================
1) Agregar libro
2) Listar libros
3) Buscar libro (título/autor/categoría)
4) Editar libro
5) Eliminar libro
6) Cambiar disponibilidad
7) Guardar
8) Cargar
0) Salir
=====================================
Seleccione una opción:
```

Ilustración 1 Menú principal del sistema en ejecución (interacción por consola)

PASO 1: Continuación del plan

Inicio de la codificación del sistema

Una vez preparado el ambiente de desarrollo, que incluyó la configuración del lenguaje Go, el editor de código y el módulo del proyecto, se continuó con la codificación del sistema seleccionado. En esta etapa se tomó como base el código desarrollado en el Aprendizaje Autónomo 1 y se procedió a mejorarlo tanto a nivel estructural como funcional.

El proceso de codificación implicó una revisión del diseño existente, identificando oportunidades de mejora en la organización del código y en la separación de responsabilidades. Esto permitió evolucionar el sistema desde una solución básica hacia un software más estructurado, ordenado y alineado con los principios de la programación orientada a objetos.

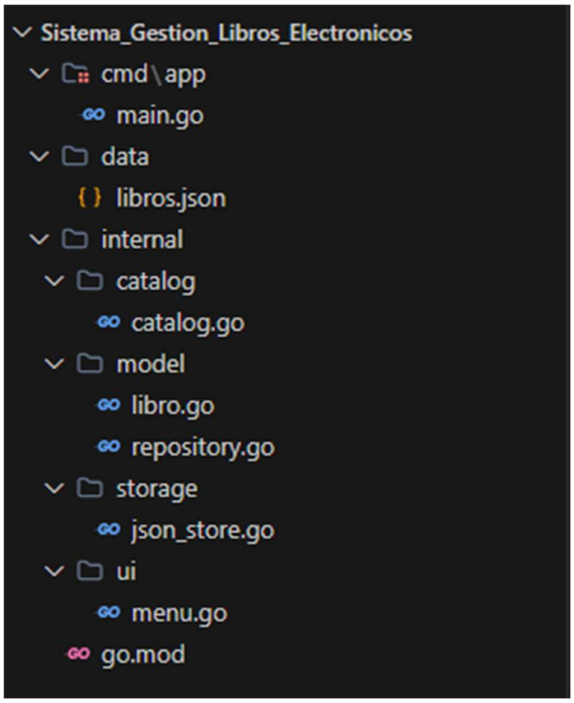

Ilustración 2 Estructura modular del proyecto y separación por capas (cmd, model, catalog, storage, ui y data).

Aplicación de los temas de las Unidades 1 y 2

En función de los temas revisados en las Unidades 1 y 2, se aplicaron conceptos fundamentales de programación orientada a objetos, tales como la identificación de entidades del dominio, la separación de responsabilidades y el uso adecuado de estructuras de datos. Para la gestión de los libros se utilizaron slices, permitiendo operaciones dinámicas como agregar, eliminar, buscar y listar elementos de forma eficiente.

Además, se adoptó una arquitectura modular basada en capas, donde cada componente del sistema cumple una función específica. Esta organización reduce el acoplamiento entre módulos y facilita la comprensión y mantenimiento del código, evidenciando una correcta aplicación de los principios orientados a objetos.

```
internal > catalog > catalog.go > NextID
25  func AddBook(books []model.Libro, nuevo model.Libro) ([]model.Libro, error) {
34      }
35      if nuevo.ID <= 0 {
36          nuevo.ID = NextID(books)
37      }
38      books = append(books, nuevo)
39      return books, nil
40  }
41
42  func ListBooks(books []model.Libro) []model.Libro {
43      // Enfoque funcional: devolvemos una copia (evita modificar el slice original por accidente)
44      out := make([]model.Libro, len(books))
45      copy(out, books)
46      return out
47  }
48
49  func SearchBooks(books []model.Libro, query string) []model.Libro {
50      q := strings.ToLower(strings.TrimSpace(query))
51      if q == "" {
52          return nil
53      }
54
55      var res []model.Libro
56      for _, b := range books {
57          if strings.Contains(strings.ToLower(b.Titulo), q) ||
58              strings.Contains(strings.ToLower(b.Autor), q) ||
59              strings.Contains(strings.ToLower(b.Categoria), q) {
60              res = append(res, b)
61          }
62      }
63      return res
64  }
65
66  func UpdateBook(books []model.Libro, id int, cambios model.Libro) ([]model.Libro, error) {
67      idx := indexByID(books, id)
68      if idx == -1 {
69          return books, ErrNotFound
70      }
71
72      // Solo actualiza campos si vienen con valor (para facilitar edición parcial)
73      if strings.TrimSpace(cambios.Titulo) != "" {
74          books[idx].Titulo = strings.TrimSpace(cambios.Titulo)
75      }
76      if strings.TrimSpace(cambios.Autor) != "" {
```

Ilustración 3 Comentarios explicativos en funcionalidades complejas para facilitar el entendimiento del código.

Identificación de clases (estructuras) y documentación de su uso

Entidad Libro

La estructura Libro fue definida como la entidad principal del sistema. En ella se concentran todos los atributos necesarios para representar un libro electrónico, como identificador, título, autor, categoría, año de publicación, formato, URL del

archivo y estado de disponibilidad. Esta consolidación permite manejar los libros como objetos coherentes dentro del sistema.

Adicionalmente, la entidad Libro incorpora comportamiento propio mediante el método String(), el cual estandariza su representación textual. Esta decisión evita que la lógica de presentación se disperse en otras capas y refuerza el uso del paradigma orientado a objetos.

```go
internal > model > libro.go > ...
package model

import "fmt"

type Libro struct {
    ID         int    `json:"id"`
    Titulo     string `json:"titulo"`
    Autor      string `json:"autor"`
    Categoria  string `json:"categoria"`
    Anio       int    `json:"anio"`
    Formato    string `json:"formato"`
    URLArchivo string `json:"urlArchivo"`
    Disponible bool   `json:"disponible"`
}

func (l Libro) String() string {
    estado := "No disponible"
    if l.Disponible {
        estado = "Disponible"
    }
    return fmt.Sprintf(
        "ID: %d | Título: %s | Autor: %s | Categoría: %s | Año: %d | Formato: %s | Archivo: %s | Estado: %s",
        l.ID, l.Titulo, l.Autor, l.Categoria, l.Anio, l.Formato, l.URLArchivo, estado,
    )
}
```

Ilustración 4 Método String() asociado a Libro para su representación textual.

Lógica de negocio (Catalog)

La lógica de negocio se concentra en el paquete catalog, encargado de gestionar la colección de libros y aplicar las reglas del sistema. Este componente define operaciones como agregar, listar, buscar, actualizar, eliminar y cambiar la disponibilidad de los libros, garantizando que todas las modificaciones se realicen de manera controlada.

Esta capa actúa como intermediaria entre la interfaz de usuario y el modelo de datos, evitando accesos directos a la colección y fortaleciendo la encapsulación del sistema.

```go
internal > catalog > catalog.go > NextID

func AddBook(books []model.Libro, nuevo model.Libro) ([]model.Libro, error) {
    nuevo.Titulo = strings.TrimSpace(nuevo.Titulo)
    nuevo.Autor = strings.TrimSpace(nuevo.Autor)
    nuevo.Categoria = strings.TrimSpace(nuevo.Categoria)
    nuevo.Formato = strings.TrimSpace(nuevo.Formato)
    nuevo.URLArchivo = strings.TrimSpace(nuevo.URLArchivo)

    if nuevo.Titulo == "" || nuevo.Autor == "" {
        return books, ErrInvalidData
    }
    if nuevo.ID <= 0 {
        nuevo.ID = NextID(books)
    }
    books = append(books, nuevo)
    return books, nil
}

func ListBooks(books []model.Libro) []model.Libro {
    // Enfoque funcional: devolvemos una copia (evita modificar el slice original por accidente)
    out := make([]model.Libro, len(books))
    copy(out, books)
    return out
}

func SearchBooks(books []model.Libro, query string) []model.Libro {
    q := strings.ToLower(strings.TrimSpace(query))
    if q == "" {
        return nil
    }

    var res []model.Libro
    for _, b := range books {
        if strings.Contains(strings.ToLower(b.Titulo), q) ||
            strings.Contains(strings.ToLower(b.Autor), q) ||
            strings.Contains(strings.ToLower(b.Categoria), q) {
            res = append(res, b)
        }
    }
    return res
}
```

Ilustración 5 Funciones del paquete catalog que gestionan el CRUD y reglas del negocio

Persistencia de datos

La persistencia de datos se implementó en el paquete storage, encargado de cargar y guardar la información en archivos JSON. Este componente maneja escenarios como archivo inexistente, archivo vacío y errores de deserialización, asegurando que el sistema sea robusto frente a fallos.

```go
internal > storage > json_store.go > ...
// Load carga los libros desde el archivo JSON.
// Si el archivo no existe o está vacío, retorna slice vacío sin error.
func (s JSONStore) Load() ([]model.Libro, error) {
    _, err := os.Stat(s.Path)
    if err != nil {
        if errors.Is(err, os.ErrNotExist) {
            return []model.Libro{}, nil
        }
        return nil, err
    }

    data, err := os.ReadFile(s.Path)
    if err != nil {
        return nil, err
    }
    if len(data) == 0 {
        return []model.Libro{}, nil
    }

    var books []model.Libro
    if err := json.Unmarshal(data, &books); err != nil {
        return nil, err
    }
    return books, nil
}
```

Ilustración 6 Manejo de errores en la carga y guardado de datos en formato JSON.

Avance significativo del desarrollo del software

El avance significativo del desarrollo se evidencia en la evolución del sistema respecto al Aprendizaje Autónomo 1. El software pasó de ser una aplicación básica a un sistema con arquitectura modular, persistencia robusta, validaciones de entrada y funcionalidades específicas del dominio, como la gestión de disponibilidad de los libros.

```
Problems    Output    Debug Console    Terminal    Ports

PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos> go run ./cmd/app
=====================================
  SISTEMA DE GESTIÓN DE E-BOOKS (GO)
=====================================
1) Agregar libro
2) Listar libros
3) Buscar libro (título/autor/categoría)
4) Editar libro
5) Eliminar libro
6) Cambiar disponibilidad
7) Guardar
8) Cargar
0) Salir
=====================================
Seleccione una opción: 1

== Agregar libro ==
Título: Caminar y Adelgazar
Autor: Charo Sierra
Categoría: Bienestar
Año (0 si no aplica): 2023
Formato (PDF/EPUB/MOBI): EPUB
URL o ruta del archivo: https://espaebook.net/book/caminar-y-adelgazar/
¿Disponible? (s/n): s
✅ Libro agregado.

Presione ENTER para continuar...
```

Ilustración 7 Evidencia de funcionamiento: registro y listado de libros con salida formateada.

PASO 2: Desarrollo del sistema

Implementación de los temas de la Unidad 3 (Interfaces)

Como parte de los contenidos de la Unidad 3, se implementaron interfaces para desacoplar la lógica del sistema del mecanismo concreto de persistencia. Se definió la interfaz BookRepository en la capa de dominio, estableciendo un contrato claro para las operaciones de carga y guardado de libros.

Posteriormente, esta interfaz fue implementada mediante la estructura JSONStore, encargada de persistir los datos en formato JSON. La interfaz es utilizada por la capa de interfaz de usuario, mientras que la implementación concreta es inyectada desde el punto de entrada del sistema. De esta manera, la UI depende de una abstracción y no de una implementación específica.

```
internal > model > repository.go > ...
1   package model
2
3   // BookRepository define un contrato para persistir libros.
4   // La UI y la lógica del sistema deben depender de esta interfaz,
5   // no de una implementación concreta (JSON, BD, etc.).
6   type BookRepository interface {
7       Load() ([]Libro, error)
8       Save(books []Libro) error
9   }
10
```

Ilustración 8 Implementación concreta de BookRepository mediante JSONStore con métodos Load() y Save().

Encapsulación

La encapsulación se aplicó evitando que la interfaz de usuario manipule directamente los datos internos del sistema. Todas las operaciones sobre la colección de libros se realizan mediante la capa de lógica de negocio, manteniendo el control sobre las modificaciones permitidas.

En el caso de la entidad Libro, los campos se mantienen exportados debido a una limitación técnica del paquete encoding/json, que solo permite serializar campos exportados. Esta decisión fue documentada explícitamente en el código, evidenciando una elección consciente de diseño y no un incumplimiento del principio de encapsulación.

```go
internal > catalog > catalog.go > NextID
func DeleteBook(books []model.Libro, id int) ([]model.Libro, error) {
    idx := indexByID(books, id)
    if idx == -1 {
        return books, ErrNotFound
    }
    // elimina sin dejar huecos
    books = append(books[:idx], books[idx+1:]...)
    return books, nil
}

func ToggleAvailability(books []model.Libro, id int) ([]model.Libro, error) {
    idx := indexByID(books, id)
    if idx == -1 {
        return books, ErrNotFound
    }
    books[idx].Disponible = !books[idx].Disponible
    return books, nil
}

func GetByID(books []model.Libro, id int) (model.Libro, error) {
    idx := indexByID(books, id)
    if idx == -1 {
        return model.Libro{}, ErrNotFound
    }
    return books[idx], nil
}

func indexByID(books []model.Libro, id int) int {
    for i, b := range books {
        if b.ID == id {
            return i
        }
    }
    return -1
}
```

Ilustración 9 Encapsulación en la capa de lógica: las operaciones sobre libros se realizan mediante funciones del paquete catalog y se restringe lógica interna con funciones no exportadas.

Manejo de errores

El sistema incorpora manejo de errores en todas las operaciones críticas. En la persistencia se controlan errores de lectura, escritura y deserialización. En la interfaz de usuario se validan las entradas y se muestran mensajes claros, evitando fallos silenciosos y mejorando la experiencia del usuario.

```go
internal > ui > menu.go > ...

func readInt(r *bufio.Reader, label string) int {
    for {
        s := readLine(r, label)
        if s == "" {
            return 0
        }
        v, err := strconv.Atoi(s)
        if err == nil {
            return v
        }
        fmt.Println("Ingrese un número válido.")
    }
}

func readBool(r *bufio.Reader, label string) bool {
    for {
        s := strings.ToLower(readLine(r, label))
        if s == "s" || s == "si" || s == "sí" || s == "y" || s == "yes" {
            return true
        }
        if s == "n" || s == "no" {
            return false
        }
        fmt.Println("Responda con s/n.")
    }
}
```

Ilustración 10 Manejo de errores y validación de entradas en la capa de interfaz de usuario (UI)

Comentarios y documentación del código

Se añadieron comentarios explicativos en las secciones más complejas del código, especialmente en aquellas relacionadas con la actualización selectiva de datos, eliminación de elementos y validación de entradas. Esto facilita la comprensión del funcionamiento interno del sistema y mejora su mantenibilidad.

Conclusiones

Durante el desarrollo del Aprendizaje Autónomo 2 logré consolidar la aplicación de los principios de programación orientada a objetos, ya que pasé de un sistema únicamente funcional a un software estructurado por capas, con responsabilidades bien definidas y un diseño más claro. La identificación de entidades, la separación entre modelo, lógica de negocio, persistencia e interfaz de usuario me permitió comprender la importancia de una arquitectura organizada para el desarrollo de sistemas mantenibles y escalables.

La implementación de interfaces representó uno de los aprendizajes más significativos de este trabajo, debido a que me permitió desacoplar la lógica del sistema del mecanismo de almacenamiento concreto. Al definir un contrato mediante la interfaz BookRepository y utilizarlo desde la interfaz de usuario, entendí cómo aplicar la abstracción y el polimorfismo en un entorno real, cumpliendo con los contenidos de la Unidad 3 y fortaleciendo mis habilidades de diseño de software.

El fortalecimiento del manejo de errores, la encapsulación y la documentación del código contribuyó directamente a la robustez y claridad del sistema, ya que ahora el programa valida entradas, controla situaciones inesperadas y explica las decisiones técnicas mediante comentarios. Este proceso me permitió desarrollar un sistema más confiable y fácil de mantener, además de reforzar la importancia de escribir código comprensible y bien documentado como parte fundamental del desarrollo profesional de software.

URL web repositorio de Github

https://github.com/Jhonna555/Jhonna555-Sistema_de_Gestion_de_libros_Electronicos.git